American International University-Bangladesh
Department of Computer Science

CSC4164

Advanced Programming

.NET / ASP.NET Core MVC

Pre-Lab Assignment

Environment Setup & First MVC App

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Assignment	Pre-Lab Assignment — Take-Home (Due before Lab 2, Week 2)
Instructor	Jahid Hassan
Duration	Self-paced — estimated 60 to 90 minutes
Submission	Bring your completed project to Lab 2
Graded	No — this is a preparation assignment only

> **Tip:** This is not a graded in-class lab session. It is a guided take-home assignment to ensure your development environment is ready before Lab 2. Work through the tasks at your own pace. If you encounter issues, note them and ask your instructor in Consulting Hours.

> **Note:** What to bring to Lab 2: A working ASP.NET Core MVC project named FirstMVCApp with StudentController and views created as instructed in this manual.

1. Lab Objectives

By the end of this assignment, you will be able to:

- Install and verify the .NET SDK and Visual Studio development environment
- Create a new ASP.NET Core MVC project from scratch
- Understand and navigate the default MVC project folder structure
- Create a Controller with multiple action methods
- Create Views and link them to controller actions
- Run the application in a browser and observe the request-response cycle

2. Prerequisites

Before starting this assignment, ensure the following:

- You have reviewed Module 0 (History of .NET) and Module 1 (MVC Architecture) from the theory class
- Your laptop or PC has at least 8 GB RAM and 20 GB free disk space
- You have administrator access to install software

3. Theory Recap

3.1 The MVC Pattern

ASP.NET Core MVC is a web framework that follows the Model-View-Controller architectural pattern. It separates an application into three distinct components, each with a clearly defined responsibility.

Component	Responsibility	Example
Model	Represents data and business logic	Student.cs — holds Name, ID, GPA
View	Handles the UI and presentation	Index.cshtml — displays a student table
Controller	Handles requests and returns responses	StudentController.cs — Index(), Details()

3.2 Project Folder Structure

When you create a new ASP.NET Core MVC project in Visual Studio, the following folders and files are generated:

Folder / File	Purpose
Controllers/	Contains controller classes that handle incoming HTTP requests
Models/	Contains data model and entity classes
Views/	Contains Razor (.cshtml) view files, organised by controller name
Views/Shared/	Contains the shared layout file (_Layout.cshtml) and partial views
wwwroot/	Static files — CSS, JavaScript, images served directly to the browser
Program.cs	Application entry point — configures middleware pipeline and services
appsettings.json	Application configuration — connection strings, app settings

4. Required Software

Software	Version	Where to Get It
.NET SDK	.NET 8 or later	https://dotnet.microsoft.com/download
Visual Studio	>=2022 Community (free)	https://visualstudio.microsoft.com
Browser	Chrome or Edge	Pre-installed on most systems

Warning: During Visual Studio installation, ensure the ASP.NET and web development workload is selected. Without it, the MVC project template will not be available.

5. Assignment Tasks

Warning: Important: Type all code manually. Do not copy and paste from this manual. Typing the code builds muscle memory, helps you spot errors, and improves your understanding of the syntax.

Task 1 Verify Your Development Environment

Confirm that Visual Studio is correctly installed and the required workload is available.

Warning: If ASP.NET and web development workload is missing, check it in the installer and click Modify. This may take several minutes depending on your internet connection.

Task 2 Create Your First ASP.NET Core MVC Project

You will create a new project using the Visual Studio project wizard.

1. Open Visual Studio and click Create a new project on the start screen
2. In the search box, type ASP.NET Core Web App (Model-View-Controller)
3. Select it from the list and click Next
4. Set the Project name to: FirstMVCApp
5. Choose a location, for example: C:\AIUB\CSC4164\
6. Click Next and ensure the Framework is set to .NET 8.0 or Greater
7. Click Create and wait for the project to be generated

Warning: Do not select Individual Accounts under Authentication type. Keep it set to None. Authentication will be added in a later lab session.

Task 3 Explore the Project Folder Structure

Before writing any code, spend time exploring the generated files and understanding their roles.

8. In Solution Explorer, expand the Controllers folder and open HomeController.cs
9. Read through the Index(), Privacy(), and Error() action methods
10. Expand the Views folder, then the Home subfolder, and open Index.cshtml
11. Read the Razor markup and observe how it differs from plain HTML
12. Open Views/Shared/_Layout.cshtml — this is the master layout applied to all pages
13. Open Program.cs and read through the middleware pipeline configuration
14. Open appsettings.json and note how configuration values are stored

> **Tip:** Your instructor will ask you in Lab 2 to explain what app.UseRouting() and app.MapControllerRoute() do in Program.cs. Make sure you understand these before coming to class.

Task 4 Create Your First Controller

You will add a new controller named StudentController with three action methods.

Step 1 — Add the controller

15. Right-click the Controllers folder in Solution Explorer
16. Select Add → Controller
17. Choose MVC Controller — Empty and click Add
18. Name it StudentController and click Add

> **Warning:** Controllers should be suffixed with “Controller” Word

Step 2 — Add action methods

Replace all existing code in StudentController.cs with the following:

```
using Microsoft.AspNetCore.Mvc;

namespace FirstMVCApp.Controllers
{
    public class StudentController : Controller
    {
        // Displays the student list page
        public IActionResult Index()
        {
            ViewBag.Title = "Student List";
            ViewBag.Message = "Welcome to the Student Module";
            return View();
        }

        // Displays details for a specific student
        public IActionResult Details(int id)
        {
```

```
            ViewBag.StudentId = id;
            return View();
        }

        // Returns plain text — no view required
        public IActionResult About()
        {
            return Content("This is the Student module.");
        }
    }
}
```

Step 3 — Test the About action

19. Press F5 to run the application
20. Navigate to /Student/About in the browser
21. Confirm the page displays: This is the Student module.

Task 5 Create Views for the Student Controller

You will create two Razor views for the Index and Details actions.

Step 1 — Create the Views/Student folder

22. Right-click the Views folder → Add → New Folder
23. Name the folder: Student

Step 2 — Create the Index view

24. Right-click the Views/Student folder → Add → View
25. Select Razor View — Empty and click Add
26. Name it Index and click Add
27. Type the following code:

```
@{
    ViewData["Title"] = "Students";
}

<h2>@ViewBag.Title</h2>
<p>@ViewBag.Message</p>

<ul>
    <li><a href="/student/details/1">Student 1</a></li>
    <li><a href="/student/details/2">Student 2</a></li>
    <li><a href="/student/details/3">Student 3</a></li>
</ul>
```

Step 3 — Create the Details view

28. Right-click the Views/Student folder → Add → View

29. Select Razor View — Empty and click Add
30. Name it Details and click Add
31. Type the following code:

```
@{
    ViewData["Title"] = "Student Details";
}

<h2>Student Details</h2>
<p>Showing details for Student ID: <strong>@ViewBag.StudentId</strong></p>

<a href="/student/index">Back to List</a>
```

Step 4 — Test navigation

1. Press F5 to run the application
2. Navigate to /Student/Index
3. Click each student link and confirm it navigates to /Student/Details/1, /2, and /3
4. Click Back to List and confirm it returns to the Index view

Task 6 Observe the Request-Response Cycle

Use the browser developer tools to observe how ASP.NET Core handles HTTP requests.

5. Run the application and open browser Developer Tools using F12
6. Go to the Network tab in Developer Tools
7. Navigate to /Student/Index and observe the HTTP GET request
8. Check the status code (200 OK) and response content type (text/html)
9. Navigate to /Student/Details/5 and observe how the route parameter id=5 is passed
10. Navigate to /Student/NonExistent and observe the 404 Not Found response

> **Note:** Key point: The URL pattern /Controller/Action/Id maps directly to the controller class name, action method name, and route parameter as defined in Program.cs.

6. Common Errors and Solutions

Error	Likely Cause	Solution
MVC project template not found	ASP.NET workload not installed	Go to Tools → Get Tools and Features, install ASP.NET and web development workload
View not found error in browser	View file in wrong folder or wrong name	Ensure view is in Views/Student/ and named Index.cshtml or Details.cshtml exactly
Build errors after adding controller	Syntax error or missing using statement	Check for missing semicolons, braces, or add using Microsoft.AspNetCore.Mvc at the top
ViewBag data shows as empty in view	Property not set before return View()	Assign ViewBag values in the controller action before calling return View()
404 Not Found on navigation	URL does not match route pattern	Confirm URL is /StudentController/ActionName or /Student/ActionName (without 'Controller')

7. Completion Checklist

Use this checklist to confirm you have completed all tasks before coming to Lab 2:

#	Checklist Item	Done
1	Visual Studio 2022 installed with ASP.NET and web development workload	☐
2	FirstMVCApp project created successfully and runs without errors	☐
3	Project folder structure explored — Controllers, Views, wwwroot, Program.cs understood	☐
4	StudentController.cs created with Index(), Details(), and About() actions	☐
5	Views/Student/Index.cshtml created and displays ViewBag data correctly	☐
6	Views/Student/Details.cshtml created and displays the student ID from the route	☐
7	Navigation between Index and Details views tested and working in browser	☐
8	Request-response cycle observed using browser Developer Tools	☐

8. Self-Assessment Questions

These questions check your conceptual understanding. Answers will be discussed at the start of Lab 2.

- What is the role of the Controller in the MVC pattern?
- What does return View() do inside a controller action method?
- What is the difference between ViewBag and ViewData?
- Where must a Razor view file be placed so that the framework can find it automatically?
- What does the default route pattern {controller}/{action}/{id?} mean in Program.cs?

> **Tip:** These questions are likely to appear in Quiz 1 (Week 4). Review them carefully.

9. Further Reading

- ASP.NET Core MVC Overview: https://learn.microsoft.com/aspnet/core/mvc/overview
- Controllers in ASP.NET Core: https://learn.microsoft.com/aspnet/core/mvc/controllers/actions
- Views in ASP.NET Core MVC: https://learn.microsoft.com/aspnet/core/mvc/views/overview

---

Complete this assignment before attending Lab 2 in Week 2.